

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF TECHNOLOGY
PANDIT DENDAYAL ENERGY UNIVERSITY
SESSION 2026-27

SUBMITTED BY

NAME	:	Patel Ishan
ROLL NO.	:	25BCP113
DIVISION	:	3
COURSE NAME	:	Database Management System
COURSE CODE	:	24CS202L

SUBMITTED TO

Dr. Sonam Nahar

Assistant Professor

Department of Computer Science and Engineering

Pandit Deendayal Energy University

****

Pandit Deendayal Energy University

Raisan, Gandhinagar - 380 007, Gujarat, India

Computer Science and Engineering

Certificate

This is to certify that

Mr./Ms. **Patel Ishan** Roll no. **25BCP113**

of 3rd semester degree course in Computer Science and Engineering has satisfactorily completed his/her term work in **Database Management System Lab (24CS202L)** subject during the semester from_____ to _____ at School of Technology, PDEU.

Date of Submission: _____

Signature:

Faculty In-charge Head of Department

INDEX

Exp. No.	Title of Lab Work	Date of Lab Work	Page	Signature
1	Dataset Creation and Updating Using File Handling Program			
2	Data Definition Language (DDL) Commands and Table Population			
3	Data Manipulation Language (DML) Commands and Table Alteration			
4	DDL Commands with Data Integrity Constraints			
5	Data Control Language (DCL) - Security, Users, GRANT and REVOKE			
6	Relational Algebra Operations, Aggregate Functions & Transaction Control			
7	Advanced Nested Queries (Subqueries on EMP and DEPT)			
8	Data Summarization Using GROUP BY and HAVING Clauses			
9	Relational JOIN Operations in SQL (Inner, Outer, Self JOINs)			
10	Advanced Database Objects: Views, Functions, Procedures and Triggers			

EXPERIMENT NO. 1

Dataset Creation and Updating Using File Handling Program

1. Objective

To design and implement a persistent file-handling program in Python to perform Create, Read, Update, and Delete (CRUD) operations on student dataset records, manually enforcing primary key uniqueness (Roll Number) and storing structured comma-separated values (CSV) suitable for subsequent relational database ingestion.

2. Plain Text Queries

1. 1. Create Record: Prompt for Roll Number, Name, Age, Course, and Marks. Verify uniqueness of Roll Number before appending record to 'students.txt'.

2. 2. Display Records: Read all lines from the file and format them into an aligned tabular representation with column headers.

3. 3. Search by Roll Number: Accept a target Roll Number from the user, search sequentially through the file, and display matching student details.

4. 4. Update Record: Provide an update sub-menu allowing modification of Name, Age, Course, or Marks for a specified Roll Number.

5. 5. Delete Record: Remove a specific student record by Roll Number, updating the persistent storage file accordingly.

6. 6. Menu System: Implement an interactive loop providing menu choices 1-6 until the user selects 'Exit'.

3. Python Source Code

```
import os

filename = "students.txt"

# Create file if it doesn't exist
if not os.path.exists(filename):
    with open(filename, "w") as f:
        pass

# 1. Add Student
def add_student():
    rollno = input("Enter Roll Number: ")
    # Check if roll number already exists
    with open(filename, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if data and data[0] == rollno:
                print("Error: Roll number already exists!")
                return
    name = input("Enter Name: ")
    age = input("Enter Age: ")
    course = input("Enter Course: ")
    marks = input("Enter Marks: ")
    with open(filename, "a") as f:
        f.write(f"{rollno},{name},{age},{course},{marks}\n")
```

```
    print("Record Added Successfully!")

# 2. Display Data
def display():
    if os.stat(filename).st_size == 0:
        print("No records found!")
        return
    print(f"\n{'Rollno':<10} | {'Name':<20} | {'Age':<5} | {'Course':<15} |
{'Marks':<10} |")
    print("-" * 75)
    with open(filename, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if len(data) == 5:
                print(f"{data[0]:<10} | {data[1]:<20} | {data[2]:<5} | {data[3]:<15}
| {data[4]:<10} |")

# 3. Search by Roll No.
def search_rollno():
    key = input("Enter Roll Number To Search: ")
    with open(filename, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if data and data[0] == key:
                print(f"\n{'Rollno':<10} | {'Name':<20} | {'Age':<5} |
{'Course':<15} | {'Marks':<10} |")
                print("-" * 75)
                print(f"{data[0]:<10} | {data[1]:<20} | {data[2]:<5} | {data[3]:<15}
| {data[4]:<10} |")
                return
    print("No records found!")

# 4. Update
def update():
    key = input("Enter Roll Number To Update: ")
    records = []
    updated = False
    with open(filename, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if data and data[0] == key:
                print("\n\tUPDATE MENU")
                print("1. Change Name")
                print("2. Change Age")
                print("3. Change Course")
                print("4. Change Marks")
                ch = input("Enter Choice (1-4): ")
                if ch == '1':
                    data[1] = input("New Name: ")
                elif ch == '2':
                    data[2] = input("New Age: ")
                elif ch == '3':
                    data[3] = input("New Course: ")
                elif ch == '4':
                    data[4] = input("New Marks: ")
                else:
                    print("Invalid choice! Keeping old data.")
```

```
                updated = True
            if len(data) == 5:
                records.append(",".join(data) + "\n")
    if updated:
        with open(filename, "w") as f:
            f.writelines(records)
        print("Record Updated Successfully!")
    else:
        print("No records found with that Roll Number!")

# 5. Delete
def delete():
    key = input("Enter Roll Number To Delete: ")
    records = []
    deleted = False
    with open(filename, "r") as f:
        for line in f:
            data = line.strip().split(",")
            if data and data[0] == key:
                deleted = True
            else:
                if line.strip():
                    records.append(line)
    if deleted:
        with open(filename, "w") as f:
            f.writelines(records)
        print("Record Deleted Successfully!")
    else:
        print("No Record found to Delete.")

# Main Menu Block
if __name__ == "__main__":
    needed = True
    while needed:
        print("\n" + "_"*25 + "MENU" + "_"*25)
        print("1. Add Student Data")
        print("2. Display Student Data")
        print("3. Search By Roll No.")
        print("4. Update Student Data")
        print("5. Delete Student Data")
        print("6. Exit")
        try:
            ch = int(input("\nEnter Choice: "))
            if ch == 1:
                add_student()
            elif ch == 2:
                display()
            elif ch == 3:
                search_rollno()
            elif ch == 4:
                update()
            elif ch == 5:
                delete()
            elif ch == 6:
                print("Exiting Program...")
                needed = False
            else:
```

```
                print("Enter a Valid Choice (1-6).")
        except ValueError:
            print("Invalid input! Please enter a number.")
```

4. Screenshot of the query output

```
PS C:\College\DBMS_Lab> python E1.py

----- STUDENT RECORD MANAGEMENT SYSTEM -----
1. Add a Student Record
2. View All Records
3. Search a Record by Roll Number
4. Update a Record
5. Delete a Record
6. Exit
Enter your choice: 1
Enter the Roll No. 25BCP113
Enter the Name: Patel Ishan
Enter the Age: 18
Enter the Department: CSE
Student record added successfully.

----- STUDENT RECORD MANAGEMENT SYSTEM -----
Enter your choice: 1
Enter the Roll No. 25BCP119
Enter the Name: Dhanya
Enter the Age: 18
Enter the Department: CSE
Student record added successfully.

----- STUDENT RECORD MANAGEMENT SYSTEM -----
Enter your choice: 1
Enter the Roll No. 25BCP113
Roll No. already exists!
```

Figure 1.1: Python interactive menu execution - Adding student records with duplicate validation (25BCP113)

```
----- STUDENT RECORD MANAGEMENT SYSTEM -----
Enter your choice: 2

Roll Number     Name                Age       Department
-----------------------------------------------------------------
25BCP113        Patel Ishan         18        CSE
25BCP119        Dhanya              18        CSE

----- STUDENT RECORD MANAGEMENT SYSTEM -----
Enter your choice: 3
Enter the Roll No.: 25BCP113

Record Found
Roll Number : 25BCP113
Name        : Patel Ishan
Age         : 18
Department  : CSE

----- STUDENT RECORD MANAGEMENT SYSTEM -----
Enter your choice: 4
Enter the Roll No. to update: 25BCP113
Enter new details:
Name: Patel Ishan
Age: 19
Department: CSE
Record Updated Successfully

----- STUDENT RECORD MANAGEMENT SYSTEM -----
Enter your choice: 6
Program terminated.
```

Figure 1.2: Displaying tabular records, searching by roll number, updating record details, and exiting

5. Take Away

Flat File Limitations: File-based persistence requires sequential linear parsing (O(N)) for searching, updating, and deleting records, lacking index structures and query optimization available in relational DBMS.

- Manual Key Uniqueness: Primary key constraints (entity integrity) must be explicitly programmed via file scans prior to writing; otherwise, accidental duplicate records can corrupt data consistency.
- File Rewriting Overhead: In-place modifications are not supported in plain text files; updating or deleting a record necessitates rewriting all records back into secondary storage.
- Interoperability with RDBMS: Delimited flat files (CSV) serve as a standard interchange format, allowing bulk loading into MySQL using SQL statements like 'LOAD DATA INFILE'.

EXPERIMENT NO. 2

Data Definition Language (DDL) Commands and Table Population

1. Objective

To design relational database schemas by creating tables with appropriate domain attributes, data types, and primary keys using DDL statements (CREATE TABLE), and populate the relations with sample records using DML statements (INSERT INTO).

2. Plain Text Queries

1. 1. Create table CLIENT_MASTER (CLIENTNO varchar(6) primary key, NAME varchar(20), ADDRESS1 varchar(30), ADDRESS2 varchar(30), CITY varchar(15), PINCODE int, STATE varchar(15), BALDUE decimal(10,2)).

2. 2. Create table PRODUCT_MASTER (PRODUCTNO varchar(6) primary key, DESCRIPTION varchar(15), PROFITPERCENT decimal(4,2), UNIT_MEASURE varchar(10), QTYONHAND int, REORDERLVL int, SELLPRICE decimal(8,2), COSTPRICE decimal(8,2)).

3. 3. Create table SALESMAN_MASTER (SALESMANNO varchar(6) primary key, SALESMANNAME varchar(20), ADDRESS1 varchar(30), ADDRESS2 varchar(30), CITY varchar(20), PINCODE int, STATE varchar(20), SALAMT real, TGTTOGET decimal(10,2), YTDSALES decimal(6,2), REMARKS varchar(60)).

4. 4. Populate CLIENT_MASTER with initial client data for Ivan bayross, Mamta muzumdar, Chhaya bankar, Ashwini joshi, Hansel colaco, and Deepak sharma.

5. 5. Populate PRODUCT_MASTER with product data including T-Shirts, Shirts, Cotton jeans, Jeans, Trousers, Pull Overs, Denim jeans, Lycra tops, and Skirts.

6. 6. Populate SALESMAN_MASTER with sales representative details for Aman, Omkar, Raj, and Ashish.

7. 7. Verify table schema structures using DESCRIBE and verify populated contents using SELECT *.

3. SQL Queries

```
-- Create CLIENT_MASTER
CREATE TABLE CLIENT_MASTER (
    CLIENTNO varchar(6) PRIMARY KEY,
    NAME varchar(20),
    ADDRESS1 varchar(30),
    ADDRESS2 varchar(30),
    CITY varchar(15),
    PINCODE int,
    STATE varchar(15),
    BALDUE decimal(10,2)
);

-- Create PRODUCT_MASTER
CREATE TABLE PRODUCT_MASTER (
    PRODUCTNO varchar(6) PRIMARY KEY,
    DESCRIPTION varchar(15),
    PROFITPERCENT decimal(4,2),
    UNIT_MEASURE varchar(10),
    QTYONHAND int,
    REORDERLVL int,
    SELLPRICE decimal(8,2),
```

```
    COSTPRICE decimal(8,2)
);

-- Create SALESMAN_MASTER
CREATE TABLE SALESMAN_MASTER (
    SALESMANNO varchar(6) PRIMARY KEY,
    SALESMANNAME varchar(20),
    ADDRESS1 varchar(30),
    ADDRESS2 varchar(30),
    CITY varchar(20),
    PINCODE int,
    STATE varchar(20),
    SALAMT real,
    TGTTOGET decimal(10,2),
    YTDSALES decimal(6,2),
    REMARKS varchar(60)
);

-- Insert Data into CLIENT_MASTER
INSERT INTO CLIENT_MASTER (CLIENTNO, NAME, CITY, PINCODE, STATE, BALDUE) VALUES
('C00001', 'Ivan bayross', 'Mumbai', 400054, 'Maharashtra', 15000),
('C00002', 'Mamta muzumdar', 'Madras', 780001, 'Tamil nadu', 0),
('C00003', 'Chhaya bankar', 'Mumbai', 400057, 'Maharashtra', 5000),
('C00004', 'Ashwini joshi', 'Bangalore', 560001, 'Karnataka', 0),
('C00005', 'Hansel colaco', 'Mumbai', 400060, 'Maharashtra', 2000),
('C00006', 'Deepak sharma', 'Mangalore', 560050, 'Karnataka', 0);

-- Insert Data into PRODUCT_MASTER
INSERT INTO PRODUCT_MASTER (PRODUCTNO, DESCRIPTION, PROFITPERCENT, UNIT_MEASURE,
QTYONHAND, REORDERLVL, SELLPRICE, COSTPRICE) VALUES
('P00001', 'T-Shirt', 5, 'Piece', 200, 50, 350, 250),
('P0345', 'Shirts', 6, 'Piece', 150, 50, 500, 350),
('P06734', 'Cotton jeans', 5, 'Piece', 100, 20, 600, 450),
('P07865', 'Jeans', 5, 'Piece', 100, 20, 750, 500),
('P07868', 'Trousers', 2, 'Piece', 150, 50, 850, 550),
('P07885', 'Pull Overs', 2.5, 'Piece', 80, 30, 700, 450),
('P07965', 'Denim jeans', 4, 'Piece', 100, 40, 350, 250),
('P07975', 'Lycra tops', 5, 'Piece', 70, 30, 300, 175),
('P08865', 'Skirts', 5, 'Piece', 75, 30, 450, 300);

-- Insert Data into SALESMAN_MASTER
INSERT INTO SALESMAN_MASTER (SALESMANNO, SALESMANNAME, ADDRESS1, ADDRESS2, CITY,
PINCODE, STATE, SALAMT, TGTTOGET, YTDSALES, REMARKS) VALUES
('S00001', 'Aman', 'A/14', 'Worli', 'Mumbai', 400002, 'Maharashtra', 3000, 100, 50,
'Good'),
('S00002', 'Omkar', '65', 'Nariman', 'Mumbai', 400001, 'Maharashtra', 3000, 200,
100, 'Good'),
('S00003', 'Raj', 'P-7', 'Bandra', 'Mumbai', 400032, 'Maharashtra', 3000, 200, 100,
'Good'),
('S00004', 'Ashish', 'A/5', 'Juhu', 'Mumbai', 400044, 'Maharashtra', 3500, 200, 150,
'Good');
```

4. Screenshot of the query output

```
mysql> DESCRIBE CLIENT_MASTER;
+----------+---------------+------+-----+---------+-------+
| Field    | Type          | Null | Key | Default | Extra |
+----------+---------------+------+-----+---------+-------+
| CLIENTNO | varchar(6)    | NO   | PRI | NULL    |       |
| NAME     | varchar(20)   | NO   |     | NULL    |       |
| ADDRESS1 | varchar(30)   | YES  |     | NULL    |       |
| ADDRESS2 | varchar(30)   | YES  |     | NULL    |       |
| CITY     | varchar(15)   | NO   |     | NULL    |       |
| PINCODE  | int           | NO   |     | NULL    |       |
| STATE    | varchar(15)   | NO   |     | NULL    |       |
| BALDUE   | decimal(10,2) | YES  |     | 0.00    |       |
+----------+---------------+------+-----+---------+-------+
8 rows in set (0.01 sec)

mysql> DESCRIBE PRODUCT_MASTER;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| PRODUCTNO     | varchar(6)   | NO   | PRI | NULL    |       |
| DESCRIPTION   | varchar(15)  | YES  |     | NULL    |       |
| PROFITPERCENT | decimal(4,2) | NO   |     | NULL    |       |
| UNIT_MEASURE  | varchar(10)  | NO   |     | NULL    |       |
| QTYONHAND     | int          | YES  |     | 0       |       |
| REORDERL_VL   | int          | YES  |     | 0       |       |
| SELLPRICE     | decimal(8,2) | NO   |     | NULL    |       |
| COSTPRICE     | decimal(8,2) | NO   |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
8 rows in set (0.01 sec)
```

Figure 2.1: Schema definition verification via DESCRIBE for CLIENT_MASTER and PRODUCT_MASTER

```
mysql> DESCRIBE SALESMAN_MASTER;
+--------------+--------------+------+-----+---------+-------+
| Field        | Type         | Null | Key | Default | Extra |
+--------------+--------------+------+-----+---------+-------+
| SALESMANNO   | varchar(6)   | NO   | PRI | NULL    |       |
| SALESMANNAME | varchar(20)  | NO   |     | NULL    |       |
| ADDRESS1     | varchar(30)  | YES  |     | NULL    |       |
| ADDRESS2     | varchar(30)  | YES  |     | NULL    |       |
| CITY         | varchar(20)  | NO   |     | NULL    |       |
| PINCODE      | int          | NO   |     | NULL    |       |
| STATE        | varchar(20)  | NO   |     | NULL    |       |
| SALAMT       | double       | YES  |     | NULL    |       |
| TGTTOGET     | decimal(10,0)| NO   |     | NULL    |       |
| YTDSALES     | double(6,2)  | YES  |     | 0.00    |       |
| REMARKS      | varchar(60)  | YES  |     | NULL    |       |
+--------------+--------------+------+-----+---------+-------+
11 rows in set (0.00 sec)
```

Figure 2.2: Schema definition verification via DESCRIBE for SALESMAN_MASTER

```
mysql> SELECT * FROM CLIENT_MASTER;
+----------+---------------+----------+----------+-----------+---------+-------------+----------+
| clientno | name          | address1 | address2 | city      | pincode | state       | baldue   |
+----------+---------------+----------+----------+-----------+---------+-------------+----------+
| C00001   | Ivan bayross  | NULL     | NULL     | Mumbai    |  400054 | Maharashtra | 15000.00 |
| C00002   | Mamta muzumdar| NULL     | NULL     | Madras    |  780001 | Tamil Nadu  |     0.00 |
| C00003   | Chhaya bankar | NULL     | NULL     | Mumbai    |  400057 | Maharashtra |  5000.00 |
| C00004   | Ashwini joshi | NULL     | NULL     | Bangalore |  560001 | Karnataka   |     0.00 |
| C00005   | Hansel colaco | NULL     | NULL     | Mumbai    |  400060 | Maharashtra |  2000.00 |
| C00006   | Deepak sharma | NULL     | NULL     | Mangalore |  560050 | Karnataka   |     0.00 |
+----------+---------------+----------+----------+-----------+---------+-------------+----------+
6 rows in set (0.00 sec)
```

Figure 2.3: Verification of populated records in CLIENT_MASTER

```
mysql> SELECT * FROM PRODUCT_MASTER;
+-----------+-------------+---------------+--------------+-----------+-------------+-----------+-----------+
| productno | description | profitpercent | unit_measure | qtyonhand | reorderl_vl | sellprice | costprice |
+-----------+-------------+---------------+--------------+-----------+-------------+-----------+-----------+
| P00001    | T-Shirt     |          5.00 | Piece        |       200 |          50 |    350.00 |    250.00 |
| P0345     | Shirts      |          6.00 | Piece        |       150 |          50 |    500.00 |    350.00 |
| P06734    | Cotton jeans|          5.00 | Piece        |       100 |          20 |    600.00 |    450.00 |
| P07865    | Jeans       |          5.00 | Piece        |       100 |          20 |    750.00 |    500.00 |
| P07868    | Trousers    |          2.00 | Piece        |       150 |          50 |    850.00 |    550.00 |
| P07885    | Pull Overs  |          2.50 | Piece        |        80 |          30 |    700.00 |    450.00 |
| P07965    | Denim jeans |          4.00 | Piece        |       100 |          40 |    350.00 |    250.00 |
| P07975    | Lycra tops  |          5.00 | Piece        |        70 |          30 |    300.00 |    175.00 |
| P08865    | Skirts      |          5.00 | Piece        |        75 |          30 |    450.00 |    300.00 |
+-----------+-------------+---------------+--------------+-----------+-------------+-----------+-----------+
9 rows in set (0.00 sec)

mysql> SELECT * FROM SALESMAN_MASTER;
+------------+-------------+----------+----------+--------+---------+-------------+--------+----------+----------+---------+
| salesmanno | salesmanname | address1 | address2 | city   | pincode | state       | salamt | tgttoget | ytdsales | remarks |
+------------+-------------+----------+----------+--------+---------+-------------+--------+----------+----------+---------+
| S00001     | Aman        | A/14     | Worli    | Mumbai |  400002 | Maharashtra |   3000 |      100 |    50.00 | Good    |
| S00002     | Omkar       | 65       | Nariman  | Mumbai |  400001 | Maharashtra |   3000 |      200 |   100.00 | Good    |
| S00003     | Raj         | P-7      | Bandra   | Mumbai |  400032 | Maharashtra |   3000 |      200 |   100.00 | Good    |
| S00004     | Ashish      | A/5      | Juhu     | Mumbai |  400044 | Maharashtra |   3500 |      200 |   150.00 | Good    |
+------------+-------------+----------+----------+--------+---------+-------------+--------+----------+----------+---------+
4 rows in set (0.00 sec)
```

Figure 2.4: Verification of populated records in PRODUCT_MASTER and SALESMAN_MASTER

5. Take Away

DDL vs DML Distinction: DDL (CREATE TABLE) defines the structural schema metadata inside the database catalog, whereas DML (INSERT INTO) manipulates the stored instances within those schemas.

- Numeric Precision for Currency: Attributes representing prices and balances (e.g. SELLPRICE, BALDUE) should always use DECIMAL(M, D) rather than approximate floating-point types (FLOAT/DOUBLE) to prevent IEEE 754 rounding discrepancies.
- Entity Integrity Enforcement: The PRIMARY KEY clause enforces entity integrity by guaranteeing that column values are unique and non-null, establishing an index on the table.
- Multi-Row Insertion Efficiency: Bulk multi-row INSERT statements reduce network round trips and transaction commit overhead compared to executing individual single-row insert queries.

EXPERIMENT NO. 3

Data Manipulation Language (DML) Commands and Table Alteration

1. Objective

To retrieve, filter, modify, and delete tuples using DML commands (SELECT, UPDATE, DELETE), and modify table schemas dynamically using DDL commands (ALTER TABLE, RENAME TABLE, DROP TABLE).

2. Plain Text Queries

1. 1. Find out the names of all the clients.
2. 2. Retrieve the entire contents of the CLIENT_MASTER table.
3. 3. Retrieve the list of names, city and state of all clients.
4. 4. List the various product descriptions available from the PRODUCT_MASTER table.
5. 5. List all clients who are located in Mumbai.
6. 6. Find the names of salesmen who have a salary equal to Rs. 3000.
7. 7a. Change the city of ClientNo 'C00005' to 'Bangalore'.
8. 7b. Change the BalDue of ClientNo 'C00001' to Rs. 1000.
9. 7c. Change the cost price of 'Trousers' to Rs. 950.00.
10. 7d. Change the city of all salesmen to Pune.
11. 8a. Delete all salesmen whose salaries are equal to Rs. 3500.
12. 8b. Delete all products where the quantity on hand is equal to 100.
13. 8c. Delete from CLIENT_MASTER where column state holds 'Tamil Nadu'.
14. 9a. Add a column called 'Telephone' of data type INT to CLIENT_MASTER.
15. 9b. Change the size of SELLPRICE column in PRODUCT_MASTER to DECIMAL(10,2).
16. 9c. Destroy table CLIENT_MASTER along with its data.
17. 9d. Rename table SALESMAN_MASTER to sman_mast.

3. SQL Queries

```
-- 1. Names of all clients
SELECT name FROM CLIENT_MASTER;

-- 2. Entire contents of CLIENT_MASTER
SELECT * FROM CLIENT_MASTER;

-- 3. Names, city, and state of all clients
SELECT name, city, state FROM CLIENT_MASTER;

-- 4. Product descriptions from PRODUCT_MASTER
SELECT description FROM PRODUCT_MASTER;

-- 5. Clients located in Mumbai
SELECT name FROM CLIENT_MASTER WHERE city = 'Mumbai';

-- 6. Salesmen with salary equal to 3000
SELECT salesmanname FROM SALESMAN_MASTER WHERE salamt = 3000;
```

```
-- 7a. Update city of Client C00005 to Bangalore
UPDATE CLIENT_MASTER SET city = 'Bangalore' WHERE clientno = 'C00005';

-- 7b. Update BalDue of Client C00001 to 1000
UPDATE CLIENT_MASTER SET baldue = 1000 WHERE clientno = 'C00001';

-- 7c. Update cost price of Trousers to 950
UPDATE PRODUCT_MASTER SET costprice = 950 WHERE description = 'trousers';

-- 7d. Update city of all salesmen to Pune
UPDATE SALESMAN_MASTER SET city = 'Pune';

-- 8a. Delete salesmen whose salary is 3500
DELETE FROM SALESMAN_MASTER WHERE salamt = 3500;

-- 8b. Delete products where quantity on hand is 100
DELETE FROM PRODUCT_MASTER WHERE qtyonhand = 100;

-- 8c. Delete clients where state is Tamil Nadu
DELETE FROM CLIENT_MASTER WHERE state = 'Tamil Nadu';

-- 9a. Add column Telephone
ALTER TABLE CLIENT_MASTER ADD Telephone INT;

-- 9b. Modify column sellprice
ALTER TABLE PRODUCT_MASTER MODIFY sellprice DECIMAL(10,2);

-- 9c. Drop table client_master
DROP TABLE CLIENT_MASTER;

-- 9d. Rename salesman_master
RENAME TABLE SALESMAN_MASTER TO sman_mast;
```

4. Screenshot of the query output

```
mysql> SELECT name FROM CLIENT_MASTER;
+----------------+
| name           |
+----------------+
| Ivan bayross   |
| Mamta muzumdar|
| Chhaya bankar  |
| Ashwini joshi  |
| Hansel colaco  |
| Deepak sharma  |
+----------------+
6 rows in set (0.00 sec)

mysql> SELECT name, city, state FROM CLIENT_MASTER;
+----------------+------------+--------------+
| name           | city       | state        |
+----------------+------------+--------------+
| Ivan bayross   | Mumbai     | Maharashtra  |
| Mamta muzumdar| Madras     | Tamil Nadu   |
| Chhaya bankar  | Mumbai     | Maharashtra  |
| Ashwini joshi  | Bangalore  | Karnataka    |
| Hansel colaco  | Mumbai     | Maharashtra  |
| Deepak sharma  | Mangalore  | Karnataka    |
+----------------+------------+--------------+
6 rows in set (0.00 sec)

mysql> SELECT * FROM CLIENT_MASTER WHERE city = 'Mumbai';
+----------+----------------+---------+----------+--------------+
| clientno | name           | city    | pincode | baldue       |
+----------+----------------+---------+----------+--------------+
| C00001   | Ivan bayross   | Mumbai |  400054 |    15000.00 |
| C00003   | Chhaya bankar  | Mumbai |  400057 |     5000.00 |
| C00005   | Hansel colaco  | Mumbai |  400060 |     2000.00 |
+----------+----------------+---------+----------+--------------+
3 rows in set (0.00 sec)

mysql> SELECT salesmanname FROM SALESMAN_MASTER WHERE salamt = 3000;
+---------------+
| salesmanname |
+---------------+
| Aman          |
| Omkar         |
| Raj           |
+---------------+
3 rows in set (0.00 sec)
```

Figure 3.1: Data retrieval operations (projection, full table scan, and WHERE filtering)

```
mysql> UPDATE CLIENT_MASTER SET city = 'Bangalore' WHERE clientno = 'C00005';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE CLIENT_MASTER SET baldue = 1000 WHERE clientno = 'C00001';
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE PRODUCT_MASTER SET costprice = 950.00 WHERE description = 'Trousers';
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE SALESMAN_MASTER SET city = 'Pune';
Query OK, 4 rows affected (0.01 sec)
Rows matched: 4  Changed: 4  Warnings: 0

mysql> SELECT salesmanname, city FROM SALESMAN_MASTER;
+--------------+------+
| salesmanname | city |
+--------------+------+
| Aman         | Pune |
| Omkar        | Pune |
| Raj          | Pune |
| Ashish       | Pune |
+--------------+------+
4 rows in set (0.00 sec)
```

Figure 3.2: Record update operations modifying city, baldue, costprice, and salesman locations

```
mysql> DELETE FROM SALESMAN_MASTER WHERE salamt = 3500;
Query OK, 1 row affected (0.01 sec)

mysql> DELETE FROM PRODUCT_MASTER WHERE qtyonhand = 100;
Query OK, 3 rows affected (0.01 sec)

mysql> DELETE FROM CLIENT_MASTER WHERE state = 'Tamil Nadu';
Query OK, 1 row affected (0.01 sec)

mysql> SELECT clientno, name, state FROM CLIENT_MASTER;
+----------+---------------+-------------+
| clientno | name          | state       |
+----------+---------------+-------------+
| C00001   | Ivan bayross  | Maharashtra |
| C00003   | Chhaya bankar | Maharashtra |
| C00004   | Ashwini joshi | Karnataka   |
| C00005   | Hansel colaco | Maharashtra |
| C00006   | Deepak sharma | Karnataka   |
+----------+---------------+-------------+
5 rows in set (0.00 sec)
```

Figure 3.3: Record deletion operations and verified table state

```
mysql> ALTER TABLE CLIENT_MASTER ADD Telephone INT;
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE PRODUCT_MASTER MODIFY sellprice DECIMAL(10,2) NOT NULL;
Query OK, 6 rows affected (0.02 sec)

mysql> DROP TABLE CLIENT_MASTER;
Query OK, 0 rows affected (0.01 sec)

mysql> RENAME TABLE SALESMAN_MASTER TO sman_mast;
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW TABLES;
+------------------+
| Tables_in_e2_db  |
+------------------+
| product_master   |
| sman_mast        |
+------------------+
2 rows in set (0.00 sec)
```

Figure 3.4: Dynamic table alterations (ALTER, DROP TABLE, RENAME TABLE)

5. Take Away

Selective vs Global Updates: Omitting the WHERE clause in an UPDATE statement affects every single record in the table. Unconditional updates should only be used when universal modification is strictly intended.

- Tuple Deletion vs Schema Deletion: DELETE removes rows from a table while preserving the table schema, indexes, and constraints. In contrast, DROP TABLE removes both the rows and the schema definition entirely from the system catalog.
- Schema Evolution via ALTER TABLE: The ALTER TABLE command enables column additions and data type modifications in an existing table without destroying pre-existing persisted data.

EXPERIMENT NO. 4

DDL Commands with Data Constraints

1. Objective

To enforce domain, entity, and referential data integrity rules using primary keys, foreign keys, CHECK constraints, and NOT NULL constraints in relational schemas.

2. Plain Text Queries

1. 1. Create junction table client_product(clientno, productno) using foreign key constraints referencing client_master_1(clientno) and product_master(productno).

2. 2. Create table CLIENT_MASTER_1 with primary key on CLIENTNO, NOT NULL on NAME, and a CHECK constraint ensuring CLIENTNO begins with 'C'.

3. 3. Alter table PRODUCT_MASTER to add a PRIMARY KEY on PRODUCTNO, add a CHECK constraint ensuring PRODUCTNO starts with 'P', and enforce NOT NULL constraints on all columns.

4. 4. Alter table SALESMAN_MASTER to add a PRIMARY KEY on SALESMANNO, add a CHECK constraint ensuring SALESMANNO starts with 'S', add CHECK constraints ensuring SALAMT != 0 and TGTTOGET != 0, and enforce NOT NULL constraints on essential attributes.

3. SQL Queries

```
-- 1. Create client_product with foreign key constraints
CREATE TABLE client_product (
    clientno varchar(6),
    productno varchar(6),
    FOREIGN KEY (clientno) REFERENCES client_master_1(clientno),
    FOREIGN KEY (productno) REFERENCES product_master(productno)
);

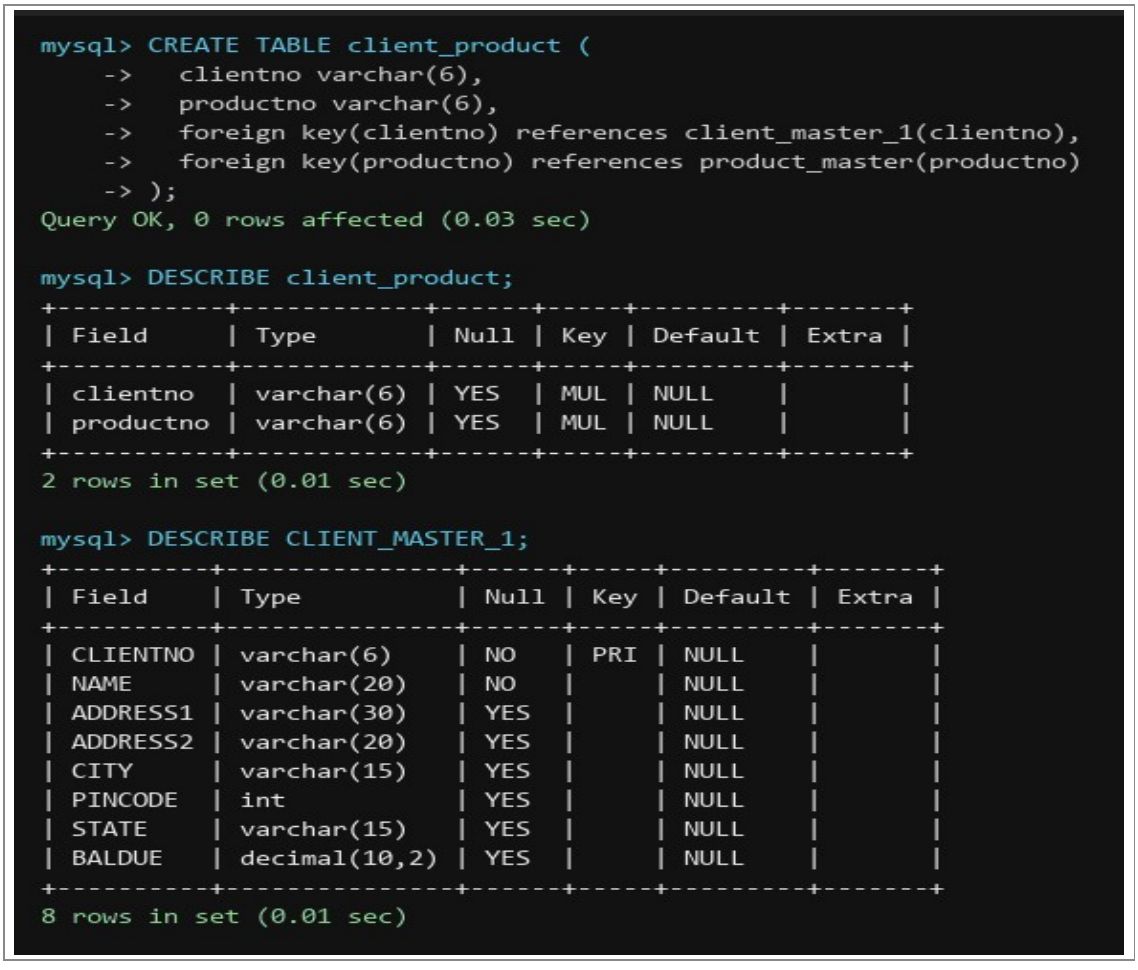
-- 2. Create CLIENT_MASTER_1 with constraints
CREATE TABLE CLIENT_MASTER_1 (
    CLIENTNO varchar(6) PRIMARY KEY,
    NAME varchar(20) NOT NULL,
    ADDRESS1 varchar(30),
    ADDRESS2 varchar(20),
    CITY varchar(15),
    PINCODE int(8),
    STATE varchar(15),
    BALDUE decimal(10,2),
    CHECK (CLIENTNO LIKE 'C%')
);

-- 3. Altering product_master for given constraints
ALTER TABLE product_master ADD PRIMARY KEY (PRODUCTNO);
ALTER TABLE product_master ADD CONSTRAINT chk_productno_p CHECK (PRODUCTNO LIKE
'P%');
ALTER TABLE product_master
    MODIFY PRODUCTNO VARCHAR(6) NOT NULL,
    MODIFY DESCRIPTION VARCHAR(15) NOT NULL,
    MODIFY PROFITPERCENT DECIMAL(4,2) NOT NULL,
    MODIFY UNIT_MEASURE VARCHAR(10) NOT NULL,
    MODIFY QTYONHAND INT NOT NULL,
    MODIFY REORDERLVL INT NOT NULL,
```

```
    MODIFY SELLPRICE DECIMAL(8,2) NOT NULL,
    MODIFY COSTPRICE DECIMAL(8,2) NOT NULL;

-- 4. Altering salesman_master for given constraints
ALTER TABLE salesman_master ADD PRIMARY KEY (SALESMANNO);
ALTER TABLE salesman_master ADD CONSTRAINT chk_salesmanno_s CHECK (SALESMANNO LIKE
'S%');
ALTER TABLE salesman_master ADD CONSTRAINT chk_salamt_not_zero CHECK (SALAMT != 0);
ALTER TABLE salesman_master ADD CONSTRAINT chk_tgttoget_not_zero CHECK (TGTTOGET !=
0);
ALTER TABLE salesman_master
    MODIFY SALESMANNAME VARCHAR(20) NOT NULL,
    MODIFY ADDRESS1 VARCHAR(30) NOT NULL,
    MODIFY SALAMT REAL NOT NULL,
    MODIFY TGTTOGET DECIMAL(10,2) NOT NULL,
    MODIFY YTDSALES DECIMAL(6,2) NOT NULL;
```

4. Screenshot of the query output



```
mysql> CREATE TABLE client_product (
    ->   clientno varchar(6),
    ->   productno varchar(6),
    ->   foreign key(clientno) references client_master_1(clientno),
    ->   foreign key(productno) references product_master(productno)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> DESCRIBE client_product;
+-----------+------------+------+-----+---------+-------+
| Field     | Type       | Null | Key | Default | Extra |
+-----------+------------+------+-----+---------+-------+
| clientno  | varchar(6) | YES  | MUL | NULL    |       |
| productno | varchar(6) | YES  | MUL | NULL    |       |
+-----------+------------+------+-----+---------+-------+
2 rows in set (0.01 sec)

mysql> DESCRIBE CLIENT_MASTER_1;
+----------+---------------+------+-----+---------+-------+
| Field    | Type          | Null | Key | Default | Extra |
+----------+---------------+------+-----+---------+-------+
| CLIENTNO | varchar(6)    | NO   | PRI | NULL    |       |
| NAME     | varchar(20)   | NO   |     | NULL    |       |
| ADDRESS1 | varchar(30)   | YES  |     | NULL    |       |
| ADDRESS2 | varchar(20)   | YES  |     | NULL    |       |
| CITY     | varchar(15)   | YES  |     | NULL    |       |
| PINCODE  | int           | YES  |     | NULL    |       |
| STATE    | varchar(15)   | YES  |     | NULL    |       |
| BALDUE   | decimal(10,2) | YES  |     | NULL    |       |
+----------+---------------+------+-----+---------+-------+
8 rows in set (0.01 sec)
```

Figure 4.1: Referential integrity in client_product (MUL keys) and CLIENT_MASTER_1 schema

```
mysql> ALTER TABLE product_master ADD PRIMARY KEY (PRODUCTNO);
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE product_master ADD CONSTRAINT chk_productno_p CHECK (PRODUCTNO LIKE 'P%');
Query OK, 9 rows affected (0.02 sec)

mysql> DESCRIBE product_master;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| PRODUCTNO     | varchar(6)   | NO   | PRI | NULL    |       |
| DESCRIPTION   | varchar(15)  | NO   |     | NULL    |       |
| PROFITPERCENT | decimal(4,2) | NO   |     | NULL    |       |
| UNIT_MEASURE  | varchar(10)  | NO   |     | NULL    |       |
| QTYONHAND     | int          | NO   |     | NULL    |       |
| REORDERLVL    | int          | NO   |     | NULL    |       |
| SELLPRICE     | decimal(8,2) | NO   |     | NULL    |       |
| COSTPRICE     | decimal(8,2) | NO   |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
8 rows in set (0.00 sec)
```

Figure 4.2: Enforcing primary keys, CHECK constraints, and NOT NULL rules on PRODUCT_MASTER

5. Take Away

Declarative Integrity at the Database Tier: Constraints act as non-bypassable database guards, ensuring data validity regardless of which application interface submits the queries.

- Referential Integrity & Foreign Keys: Foreign key constraints prevent orphaned records in child tables by rejecting child insertions whose foreign keys do not exist in the referenced parent relation.
- Domain Validation using CHECK: Pattern-matching CHECK constraints (e.g. LIKE 'C%') enforce business formatting rules directly in the relational engine, preventing malformed identifiers.

EXPERIMENT NO. 5

Data Control Language (DCL) - Security, Users, GRANT and REVOKE

1. Objective

To understand and implement database security administration by configuring user accounts, granting granular privileges with GRANT, revoking access with REVOKE, and inspecting active privileges with SHOW GRANTS.

2. Plain Text Queries

1. 1. Create database library and create table AUTHOR = {Author_Id char(5) PK, LastName varchar(15) NOT NULL, FirstName varchar(15) NOT NULL, Email varchar(40), City varchar(15), Country varchar(15)}.

2. 2. Create table BOOK = {Book_Id char(5) PK (starts with 'B'), Book_Title varchar(50) NOT NULL, Copies int (Copies > 2)}.

3. 3. Create table AUTHOR_LIST = {Author_Id char(5), Book_Id char(5), Role varchar(15)} with composite primary key (Author_Id, Book_Id) and foreign keys referencing AUTHOR and BOOK.

4. 4. Populate four records into each of the tables: AUTHOR, BOOK, and AUTHOR_LIST.

5. 5. Alter table AUTHOR_LIST to add column Publisher varchar(30).

6. 6. Create two new database users: 'manager_user'@'localhost' and 'staff_user'@'localhost'.

7. 7. Grant SELECT, INSERT, and UPDATE privileges on AUTHOR table to 'manager_user'@'localhost'.

8. 8. Grant only SELECT privilege on BOOK table to 'staff_user'@'localhost'.

9. 9. Revoke the INSERT privilege from 'manager_user'@'localhost' and verify active permissions using SHOW GRANTS.

3. SQL Queries

```
-- Create and use database
CREATE DATABASE IF NOT EXISTS library;
USE library;

-- 1. Create table AUTHOR
CREATE TABLE AUTHOR (
    Author_Id CHAR(5) PRIMARY KEY,
    LastName VARCHAR(15) NOT NULL,
    FirstName VARCHAR(15) NOT NULL,
    Email VARCHAR(40),
    City VARCHAR(15),
    Country VARCHAR(15)
);

-- 2. Create table BOOK with CHECK constraints
CREATE TABLE BOOK (
    Book_Id CHAR(5) PRIMARY KEY,
    Book_Title VARCHAR(50) NOT NULL,
    Copies INT,
    CONSTRAINT bookId_starts_with_b CHECK (Book_Id LIKE 'B%'),
    CONSTRAINT copies_more_than_2 CHECK (Copies > 2)
);
```

```
-- 3. Create table AUTHOR_LIST with foreign keys
CREATE TABLE AUTHOR_LIST (
    Author_Id CHAR(5),
    Book_Id CHAR(5),
    Role VARCHAR(15),
    PRIMARY KEY (Author_Id, Book_Id),
    FOREIGN KEY (Author_Id) REFERENCES AUTHOR(Author_Id),
    FOREIGN KEY (Book_Id) REFERENCES BOOK(Book_Id)
);

-- 4. Add four records in each table
INSERT INTO AUTHOR VALUES
('A1201', 'Holly', 'Jackson', 'hollyjackson@gmail.com', 'London', 'UK'),
('A1202', 'Sudha', 'Murty', 'sudhatinfosys@gmail.com', 'Pune', 'India'),
('A1203', 'JK', 'Rowling', 'jkrollinghp@gmail.com', 'London', 'UK'),
('A1204', 'Soph', 'Emp', 'mudernnextdoor@gmail.com', 'Toronto', 'Canada');

INSERT INTO BOOK VALUES
('B001', 'AGGGTM', 4),
('B002', 'How I taught my mother to read', 3),
('B003', 'Purple Hyacinth', 4),
('B004', 'Harry Potter-complete', 7);

INSERT INTO AUTHOR_LIST (Author_Id, Book_Id, Role, Publisher) VALUES
('A1201', 'B001', 'Fiction', NULL),
('A1202', 'B002', 'Non-Fiction', NULL),
('A1203', 'B004', 'Complete', NULL),
('A1204', 'B003', 'Graphic Novel', NULL);

-- 5. Alter table to add Publisher column
ALTER TABLE AUTHOR_LIST ADD Publisher VARCHAR(30);

-- 6. Create users
CREATE USER 'manager_user'@'localhost' IDENTIFIED BY 'Password123!';
CREATE USER 'staff_user'@'localhost' IDENTIFIED BY 'Password123!';

-- 7. Grant privileges to manager_user
GRANT SELECT, INSERT, UPDATE ON AUTHOR TO 'manager_user'@'localhost';

-- 8. Grant SELECT privileges to staff_user
GRANT SELECT ON BOOK TO 'staff_user'@'localhost';

-- 9. Revoke INSERT privilege and verify
REVOKE INSERT ON AUTHOR FROM 'manager_user'@'localhost';
SHOW GRANTS FOR 'manager_user'@'localhost';
```

4. Screenshot of the query output

```
mysql> SELECT * FROM AUTHOR;
+-----------+----------+-----------+------------------+----------+---------+
| Author_ID | Lastname | Firstname | Email            | City     | Country |
+-----------+----------+-----------+------------------+----------+---------+
| A0001     | Smith    | John      | john@example.com | New York | USA     |
| A0002     | Doe      | Jane      | jane@example.com | London   | UK      |
| A0003     | Patel    | Raj       | raj@example.com  | Mumbai   | India   |
| A0004     | Wong     | Lisa      | lisa@example.com | Toronto  | Canada  |
+-----------+----------+-----------+------------------+----------+---------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM BOOK;
+---------+--------------------+--------+
| Book_ID | Book_Title         | Copies |
+---------+--------------------+--------+
| B0001   | Database Systems   |      5 |
| B0002   | Learn SQL          |      3 |
| B0003   | Data Science 101   |     10 |
| B0004   | Advanced Computing |      4 |
+---------+--------------------+--------+
4 rows in set (0.00 sec)
```

Figure 5.1: Library database schema verification and populated AUTHOR and BOOK relations

```
mysql> SELECT * FROM AUTHOR_LIST;
+-----------+---------+-----------+-----------+
| Author_ID | Book_ID | Role      | Publisher |
+-----------+---------+-----------+-----------+
| A0001     | B0001   | Co-Author | Pearson   |
| A0002     | B0002   | Author    | O'Reilly  |
| A0003     | B0003   | Author    | Wiley     |
| A0004     | B0004   | Editor    | Springer  |
+-----------+---------+-----------+-----------+
4 rows in set (0.00 sec)
```

Figure 5.2: AUTHOR_LIST relation populated and altered with Publisher attribute

```
mysql> CREATE USER 'manager_user'@'localhost' IDENTIFIED BY 'manager123';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT SELECT, INSERT, UPDATE ON AUTHOR TO 'manager_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT ON BOOK TO 'staff_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> REVOKE INSERT ON AUTHOR FROM 'manager_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW GRANTS FOR 'manager_user'@'localhost';
+-----------------------------------------------------------------------------+
| Grants for manager_user@localhost                                           |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `manager_user`@`localhost`                            |
| GRANT SELECT, UPDATE ON `library_lab`.`AUTHOR` TO `manager_user`@`localhost`|
+-----------------------------------------------------------------------------+
2 rows in set (0.00 sec)
```

Figure 5.3: User administration: CREATE USER, GRANT privileges, REVOKE, and SHOW GRANTS

5. Take Away

Principle of Least Privilege (PoLP): Users must only be granted the minimum necessary permissions required to fulfill their roles (e.g. staff_user receives only read-only SELECT permissions).

- Granular Privilege Scoping: MySQL allows privileges to be assigned at global (*.*), database (db.*), table (db.table), or even column levels.
- Auditing Privileges with SHOW GRANTS: Database administrators must routinely audit active permissions using SHOW GRANTS to ensure that obsolete or elevated privileges (like INSERT) are removed.

EXPERIMENT NO. 6

Relational Algebra Operations, Aggregate Functions and Transaction Control

1. Objective

To execute relational algebra operations (Selection, Projection, Cartesian Product, Set Difference, and Intersection) and aggregate functions on Supplier-Part schemas, and demonstrate transaction control mechanisms using COMMIT, ROLLBACK, and SAVEPOINT.

2. Plain Text Queries

1. 1. Create tables Supplier(scode, sname, scity, turnover), Part(pcode, weight, color, cost, sellingprice), and Supplier_Part(scode, pcode, qty) and populate sample data.

2. 2. Write appropriate SQL statements for the following:

3. Q1: Get supplier number and part number in ascending order of supplier number.
4. Q2: Get details of supplier who operate from Bombay with turnover > 50.
5. Q3: Get the total number of suppliers.
6. Q4: Get the part number weighing between 25 and 35.
7. Q5: Get the supplier number whose turnover is null.
8. Q6: Get the part number that costs 20, 30 or 40 rupees.
9. Q7: Get the total quantity of part 2 that is supplied.
10. Q8: Get the name of supplier who supply part 2.
11. Q9: Get the part number whose cost is greater than the average cost.
12. Q10: Get the supplier number and turnover in descending order of turnover.
13. Q11: Find the total quantity supplied for each part.
14. Q12: Display parts whose total quantity is more than 20.
15. Q13: Return name of suppliers who have not supplied any part.
16. Q14: Find the suppliers who have supplied both parts p1 and p2.

17. 3. Demonstrate transaction control commands (START TRANSACTION, SAVEPOINT, ROLLBACK TO, COMMIT).

3. SQL Queries

```
-- Create Database and Tables
CREATE DATABASE IF NOT EXISTS lab5;
USE lab5;

CREATE TABLE Supplier (
    scode VARCHAR(5) PRIMARY KEY,
    sname VARCHAR(20),
    scity VARCHAR(10),
    turnover INT
);

CREATE TABLE Part (
    pcode VARCHAR(5) PRIMARY KEY,
    weight INT,
    color VARCHAR(20),
```

```
    cost INT,
    sellingprice INT
);

CREATE TABLE Supplier_Part (
    scode VARCHAR(5),
    pcode VARCHAR(5),
    qty INT,
    PRIMARY KEY (scode, pcode),
    FOREIGN KEY (scode) REFERENCES Supplier(scode),
    FOREIGN KEY (pcode) REFERENCES Part(pcode)
);

-- Populate Tables
INSERT INTO Supplier VALUES
('1', 'TechCorp', 'Bombay', 50),
('2', 'GlobalSupplies', 'Delhi', NULL),
('3', 'MahaParts', 'Bombay', 100);

INSERT INTO Part VALUES
('1', 30, 'Red', 20, 25),
('2', 45, 'Blue', 35, 50),
('3', 28, 'Green', 30, 40);

INSERT INTO Supplier_Part VALUES
('1', '1', 100),
('1', '2', 50),
('2', '2', 200);

-- Relational Algebra & Function Queries
-- Q1: Order by scode
SELECT * FROM Supplier_part ORDER BY scode;

-- Q2: Bombay suppliers with turnover > 50
SELECT * FROM Supplier WHERE scity = 'Bombay' AND turnover > 50;

-- Q3: Count of suppliers
SELECT COUNT(scode) FROM Supplier;

-- Q4: Weight between 25 and 35
SELECT pcode FROM Part WHERE weight BETWEEN 25 AND 35;

-- Q5: Turnover is NULL
SELECT scode FROM Supplier WHERE turnover IS NULL;

-- Q6: Cost in (20, 30, 40)
SELECT pcode FROM Part WHERE cost IN (20, 30, 40);

-- Q7: Sum of qty for part 2
SELECT SUM(qty) FROM Supplier_part WHERE pcode = 2;

-- Q8: Suppliers supplying part 2
SELECT sname FROM Supplier WHERE scode IN (SELECT scode FROM Supplier_part WHERE
pcode = 2);

-- Q9: Cost greater than average cost
```

```
SELECT pcode FROM Part WHERE cost > (SELECT AVG(cost) FROM Part);

-- Q10: Turnover in descending order
SELECT scode, turnover FROM Supplier ORDER BY turnover DESC;

-- Q11: Total quantity supplied for each part
SELECT pcode, SUM(qty) FROM Supplier_part GROUP BY pcode;

-- Q12: Parts with total quantity > 20
SELECT pcode, SUM(qty) FROM Supplier_part GROUP BY pcode HAVING SUM(qty) > 20;

-- Q13: Suppliers who have not supplied any part
SELECT sname FROM Supplier WHERE scode NOT IN (SELECT scode FROM Supplier_part);

-- Q14: Suppliers supplying both parts 1 and 2 (INTERSECT)
SELECT sname FROM Supplier WHERE scode = (
    (SELECT scode FROM Supplier_part WHERE pcode = 1)
    INTERSECT
    (SELECT scode FROM Supplier_part WHERE pcode = 2)
);

-- Transaction Control Commands Demonstration
START TRANSACTION;
INSERT INTO Supplier VALUES ('4', 'ApexTraders', 'Pune', 120);
SAVEPOINT sp1;
UPDATE Supplier SET turnover = 150 WHERE scode = '4';
ROLLBACK TO sp1;
COMMIT;
```

4. Screenshot of the query output



```
mysql> SELECT * FROM Supplier;
+-------+-----------------+--------+----------+
| scode | sname           | scity  | turnover |
+-------+-----------------+--------+----------+
|     1 | Acme Corp       | Bombay |       50 |
|     2 | Global Supplies | Delhi  |     NULL |
|     3 | TechParts Inc   | Bombay |       75 |
|     4 | Reliable Co     | Madras |     NULL |
|     5 | Sunrise Traders | Bombay |       50 |
|     6 | Apex Systems    | Pune   |      120 |
+-------+-----------------+--------+----------+
6 rows in set (0.00 sec)
```

Figure 6.1: Supplier-Part database: populated Supplier relation instances

```
mysql> SELECT scode, pcode FROM Supplier_Part ORDER BY scode ASC;
+-------+-------+
| scode | pcode |
+-------+-------+
|     1 |     1 |
|     1 |     2 |
|     2 |     1 |
|     3 |     1 |
|     3 |     2 |
|     3 |     3 |
|     5 |     4 |
|     6 |     5 |
|     6 |     6 |
+-------+-------+
9 rows in set (0.00 sec)

mysql> SELECT * FROM Supplier WHERE scity = 'Bombay' AND turnover = 50;
+-------+-----------------+--------+----------+
| scode | sname           | scity  | turnover |
+-------+-----------------+--------+----------+
|     1 | Acme Corp       | Bombay |       50 |
|     5 | Sunrise Traders | Bombay |       50 |
+-------+-----------------+--------+----------+
2 rows in set (0.00 sec)

mysql> SELECT COUNT(*) FROM Supplier;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT pcode FROM Part WHERE weigh BETWEEN 25 AND 35;
+-------+
| pcode |
+-------+
|     1 |
|     3 |
|     5 |
|     6 |
+-------+
4 rows in set (0.00 sec)
```

Figure 6.2: Queries 1 to 4: ORDER BY, WHERE filtering, COUNT aggregate, and BETWEEN range

```
mysql> SELECT scode FROM Supplier WHERE turnover IS NULL;
+-------+
| scode |
+-------+
|     2 |
|     4 |
+-------+
2 rows in set (0.00 sec)

mysql> SELECT pcode FROM Part WHERE cost IN (20, 30, 40);
+-------+
| pcode |
+-------+
|     1 |
|     3 |
|     4 |
|     6 |
+-------+
4 rows in set (0.00 sec)

mysql> SELECT SUM(qty) FROM Supplier_Part WHERE pcode = 2;
+----------+
| SUM(qty) |
+----------+
|      175 |
+----------+
1 row in set (0.00 sec)

mysql> SELECT s.sname FROM Supplier s JOIN Supplier_Part sp ON s.scode = sp.scode WHERE sp.pcode = 2;
+---------------+
| sname         |
+---------------+
| Acme Corp     |
| TechParts Inc |
+---------------+
2 rows in set (0.00 sec)
```

Figure 6.3: Queries 5 to 8: NULL turnover, IN set comparison, SUM aggregate, and join lookup

```
mysql> SELECT pcode, SUM(qty) AS total_quantity FROM Supplier_Part GROUP BY pcode;
+-------+----------------+
| pcode | total_quantity |
+-------+----------------+
|     1 |            145 |
|     2 |            175 |
|     3 |             50 |
|     4 |             10 |
|     5 |            200 |
|     6 |             75 |
+-------+----------------+
6 rows in set (0.00 sec)

mysql> SELECT pcode FROM Supplier_Part GROUP BY pcode HAVING SUM(qty) > 20;
+-------+
| pcode |
+-------+
|     1 |
|     2 |
|     3 |
|     5 |
|     6 |
+-------+
5 rows in set (0.00 sec)

mysql> SELECT sname FROM Supplier WHERE scode NOT IN (SELECT scode FROM Supplier_Part);
+-------------+
| sname       |
+-------------+
| Reliable Co |
+-------------+
1 row in set (0.00 sec)
```

Figure 6.4: Queries 11 to 13: GROUP BY summarization, HAVING filter, and NOT IN difference

5. Take Away

Aggregate Functions & NULL Handling: Aggregate functions like SUM, AVG, and COUNT(column) ignore NULL values during computation, whereas COUNT(*) counts all rows regardless of NULL presence.

- HAVING vs WHERE Execution Flow: The WHERE clause filters base rows prior to grouping, while the HAVING clause filters summary groups post-aggregation.
- Relational Completeness: Relational algebra operations (Selection, Projection, Join, Union, Intersection, Difference) map directly into declarative SQL constructs, proving the mathematical completeness of SQL.
- Transaction Atomicity via SAVEPOINT: Using SAVEPOINTS allows developers to implement granular error handling in complex transactions, rolling back flawed intermediate statements without discarding the overall transaction.

EXPERIMENT NO. 7

Advanced Nested Queries (Subqueries)

1. Objective

To design and execute single-row, multi-row, and correlated subqueries on the classic EMP and DEPT database schema to solve complex hierarchical and comparative business queries.

2. Plain Text Queries

1. 1. List the details of the employees whose salaries are more than the employee BLAKE.
2. 2. List the employees whose jobs are the same as ALLEN.
3. 3. List the employees whose salary is the same as FORD or SMITH in descending order of Names.
4. 4. List the employees whose jobs are the same as MILLER or salary is more than ALLEN.
5. 5. Find the highest paid employee of the Sales department.
6. 6. List the employees who are senior to the most recently hired employee working under KING.
7. 7. List the names of the employees who are getting the highest salary department-wise.
8. 8. List the employees whose salary is equal to the average of maximum and minimum salary.
9. 9. List the employees who joined the company on the same date.
10. 10. Find out the employees who joined the company before their Managers.

3. SQL Queries

```
-- Setup Tables and Schema
CREATE DATABASE IF NOT EXISTS lab7;
USE lab7;

CREATE TABLE EMP (
    EMPNO INT PRIMARY KEY,
    ENAME VARCHAR(20),
    JOB VARCHAR(20),
    MGR INT,
    HIREDATE DATE,
    SAL INT,
    COMM INT,
    DEPTNO INT
);

CREATE TABLE DEPT (
    DEPTNO INT PRIMARY KEY,
    DNAME VARCHAR(20),
    LOC VARCHAR(20)
);

-- Foreign Key Constraints
ALTER TABLE EMP ADD FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO);
ALTER TABLE EMP ADD FOREIGN KEY (MGR) REFERENCES EMP(EMPNO);

-- Q1: Salary > BLAKE
SELECT * FROM EMP
WHERE SAL > (SELECT SAL FROM EMP WHERE ENAME = 'BLAKE');
```

```
-- Q2: Job same as ALLEN
SELECT * FROM EMP
WHERE JOB = (SELECT JOB FROM EMP WHERE ENAME = 'ALLEN');

-- Q3: Salary = FORD or SMITH (descending name)
SELECT * FROM EMP
WHERE SAL = ANY (SELECT SAL FROM EMP WHERE ENAME IN ('FORD', 'SMITH'))
ORDER BY ENAME DESC;

-- Q4: Job = MILLER or Salary > ALLEN
SELECT * FROM EMP
WHERE JOB = (SELECT JOB FROM EMP WHERE ENAME = 'MILLER')
   OR SAL > (SELECT SAL FROM EMP WHERE ENAME = 'ALLEN');

-- Q5: Highest paid employee of Sales department
SELECT * FROM EMP
WHERE SAL = (SELECT MAX(SAL) FROM EMP WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE
DNAME = 'SALES'));

-- Q6: Senior to most recently hired employee working under KING
SELECT * FROM EMP
WHERE HIREDATE < (SELECT MAX(HIREDATE) FROM EMP WHERE MGR = (SELECT EMPNO FROM EMP
WHERE ENAME = 'KING'));

-- Q7: Highest salary department-wise
SELECT * FROM EMP
WHERE SAL IN (SELECT MAX(SAL) FROM EMP GROUP BY DEPTNO);

-- Q8: Salary equal to average of max and min salary
SELECT * FROM EMP
WHERE SAL = (((SELECT MAX(SAL) FROM EMP) + (SELECT MIN(SAL) FROM EMP)) / 2);

-- Q9: Employees who joined on the same date
SELECT * FROM EMP
WHERE HIREDATE IN (SELECT HIREDATE FROM EMP GROUP BY HIREDATE HAVING COUNT(*) > 1);

-- Q10: Employees who joined before their Managers (Correlated Subquery)
SELECT * FROM EMP e
WHERE HIREDATE < (SELECT HIREDATE FROM EMP WHERE EMPNO = e.MGR);
```

4. Screenshot of the query output

```
mysql> SELECT * FROM EMP WHERE SAL > (SELECT SAL FROM EMP WHERE ENAME = 'BLAKE');
+-------+-------+-----------+------+------------+---------+------+--------+
| EMPNO | ENAME | JOB       | MGR  | HIREDATE   | SAL     | COMM | DEPTNO |
+-------+-------+-----------+------+------------+---------+------+--------+
|  7566 | JONES | MANAGER   | 7839 | 1981-04-02 | 2975.00 | NULL |     20 |
|  7788 | SCOTT | ANALYST   | 7566 | 1982-12-09 | 3000.00 | NULL |     20 |
|  7839 | KING  | PRESIDENT | NULL | 1981-11-17 | 5000.00 | NULL |     10 |
|  7902 | FORD  | ANALYST   | 7566 | 1981-12-03 | 3000.00 | NULL |     20 |
+-------+-------+-----------+------+------------+---------+------+--------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM EMP WHERE JOB = (SELECT JOB FROM EMP WHERE ENAME = 'ALLEN');
+-------+--------+----------+------+------------+---------+---------+--------+
| EMPNO | ENAME  | JOB      | MGR  | HIREDATE   | SAL     | COMM    | DEPTNO |
+-------+--------+----------+------+------------+---------+---------+--------+
|  7499 | ALLEN  | SALESMAN | 7698 | 1981-02-20 | 1600.00 |  300.00 |     30 |
|  7521 | WARD   | SALESMAN | 7698 | 1981-02-22 | 1250.00 |  500.00 |     30 |
|  7654 | MARTIN | SALESMAN | 7698 | 1981-09-28 | 1250.00 | 1400.00 |     30 |
|  7844 | TURNER | SALESMAN | 7698 | 1981-09-08 | 1500.00 |    0.00 |     30 |
+-------+--------+----------+------+------------+---------+---------+--------+
4 rows in set (0.00 sec)
```

Figure 7.1: Nested subqueries: Salary comparison against BLAKE and Job match against ALLEN

```
mysql> SELECT * FROM EMP WHERE SAL IN (SELECT SAL FROM EMP WHERE ENAME IN ('FORD', 'SMITH')) ORDER BY ENAME DESC;
+-------+-------+---------+------+------------+---------+--------+--------+
| EMPNO | ENAME | JOB     | MGR  | HIREDATE   | SAL     | COMM   | DEPTNO |
+-------+-------+---------+------+------------+---------+--------+--------+
|  7788 | SCOTT | ANALYST | 7566 | 1982-12-09 | 3000.00 |   NULL |     20 |
|  7369 | SMITH | CLERK   | 7902 | 1980-12-17 |  500.00 | 800.00 |     20 |
|  7902 | FORD  | ANALYST | 7566 | 1981-12-03 | 3000.00 |   NULL |     20 |
+-------+-------+---------+------+------------+---------+--------+--------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM EMP WHERE JOB = (SELECT JOB FROM EMP WHERE ENAME = 'MILLER') OR SAL > (SELECT SAL FROM EMP WHERE ENAME = 'ALLEN');
+-------+--------+-----------+------+------------+---------+---------+--------+
| EMPNO | ENAME  | JOB       | MGR  | HIREDATE   | SAL     | COMM    | DEPTNO |
+-------+--------+-----------+------+------------+---------+---------+--------+
|  7369 | SMITH  | CLERK     | 7902 | 1980-12-17 |  500.00 |  800.00 |     20 |
|  7566 | JONES  | MANAGER   | 7839 | 1981-04-02 | 2975.00 |    NULL |     20 |
|  7698 | BLAKE  | MANAGER   | 7839 | 1981-05-01 | 2850.00 |    NULL |     30 |
|  7782 | CLARK  | MANAGER   | 7839 | 1981-06-09 | 2450.00 |    NULL |     10 |
|  7788 | SCOTT  | ANALYST   | 7566 | 1982-12-09 | 3000.00 |    NULL |     20 |
|  7839 | KING   | PRESIDENT | NULL | 1981-11-17 | 5000.00 |    NULL |     10 |
|  7876 | ADAMS  | CLERK     | 7788 | 1983-01-12 | 1100.00 |    NULL |     20 |
|  7900 | JAMES  | CLERK     | 7698 | 1981-12-03 |  950.00 |    NULL |     30 |
|  7902 | FORD   | ANALYST   | 7566 | 1981-12-03 | 3000.00 |    NULL |     20 |
|  7934 | MILLER | CLERK     | 7782 | 1982-01-23 | 1300.00 |    NULL |     10 |
+-------+--------+-----------+------+------------+---------+---------+--------+
10 rows in set (0.00 sec)
```

Figure 7.2: Multi-row subquery using IN and compound condition (MILLER job or ALLEN salary)

```
mysql> SELECT * FROM EMP WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE DNAME = 'SALES')
    ->   AND SAL = (SELECT MAX(SAL) FROM EMP WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE DNAME = 'SALES'));
+-------+-------+---------+------+------------+---------+------+--------+
| EMPNO | ENAME | JOB     | MGR  | HIREDATE   | SAL     | COMM | DEPTNO |
+-------+-------+---------+------+------------+---------+------+--------+
|  7698 | BLAKE | MANAGER | 7839 | 1981-05-01 | 2850.00 | NULL |     30 |
+-------+-------+---------+------+------------+---------+------+--------+
1 row in set (0.00 sec)

mysql> SELECT ENAME FROM EMP WHERE (DEPTNO, SAL) IN (SELECT DEPTNO, MAX(SAL) FROM EMP GROUP BY DEPTNO);
+-------+
| ENAME |
+-------+
| KING  |
| BLAKE |
| SCOTT |
| FORD  |
+-------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM EMP WHERE SAL = (SELECT (MAX(SAL) + MIN(SAL)) / 2 FROM EMP);
Empty set (0.00 sec)

mysql> SELECT * FROM EMP WHERE HIREDATE IN (SELECT HIREDATE FROM EMP GROUP BY HIREDATE HAVING COUNT(*) > 1);
+-------+-------+---------+------+------------+---------+------+--------+
| EMPNO | ENAME | JOB     | MGR  | HIREDATE   | SAL     | COMM | DEPTNO |
+-------+-------+---------+------+------------+---------+------+--------+
|  7900 | JAMES | CLERK   | 7698 | 1981-12-03 |  950.00 | NULL |     30 |
|  7902 | FORD  | ANALYST | 7566 | 1981-12-03 | 3000.00 | NULL |     20 |
+-------+-------+---------+------+------------+---------+------+--------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM EMP E1 WHERE HIREDATE < (SELECT HIREDATE FROM EMP E2 WHERE E1.MGR = E2.EMPNO);
+-------+-------+----------+------+------------+---------+--------+--------+
| EMPNO | ENAME | JOB      | MGR  | HIREDATE   | SAL     | COMM   | DEPTNO |
+-------+-------+----------+------+------------+---------+--------+--------+
|  7369 | SMITH | CLERK    | 7902 | 1980-12-17 |  500.00 | 800.00 |     20 |
|  7499 | ALLEN | SALESMAN | 7698 | 1981-02-20 | 1600.00 | 300.00 |     30 |
|  7521 | WARD  | SALESMAN | 7698 | 1981-02-22 | 1250.00 | 500.00 |     30 |
|  7566 | JONES | MANAGER  | 7839 | 1981-04-02 | 2975.00 |   NULL |     20 |
|  7698 | BLAKE | MANAGER  | 7839 | 1981-05-01 | 2850.00 |   NULL |     30 |
|  7782 | CLARK | MANAGER  | 7839 | 1981-06-09 | 2450.00 |   NULL |     10 |
+-------+-------+----------+------+------------+---------+--------+--------+
6 rows in set (0.00 sec)
```

Figure 7.3: Complex subqueries: Sales highest earner, dept-wise max, duplicate hire dates, and manager seniority

5. Take Away

Correlated vs Uncorrelated Subqueries: Uncorrelated subqueries execute once and pass their static result to the outer query, whereas correlated subqueries reference outer row attributes (e.g. e.MGR), executing once per candidate tuple.

- Multi-Row Evaluation with ANY and IN: Single-row comparison operators (=, <, >) throw errors when the subquery yields multiple tuples; set comparison operators (= ANY, IN) must be utilized for multi-row subqueries.
- Self-Referencing Relational Trees: The MGR attribute is a recursive foreign key referencing EMPNO within the same relation, modeling hierarchical organizational structures.

EXPERIMENT NO. 8

Data Summarization Using GROUP BY and HAVING Clauses

1. Objective

To aggregate multi-row relational data into grouped summary statistics using GROUP BY, filter grouped records using HAVING, and calculate advanced group metrics on EMP and DEPT tables.

2. Plain Text Queries

1. 1. List the deptno where there are no employees.
2. 2. List the number of employees and average salary within each department for each job.
3. 3. Find the maximum average salary drawn for each job except for 'President'.
4. 4. List the department details where at least two employees are working.
5. 5. List the number of employees in each department where the number is more than 3.
6. 6. List the names of the employees who are getting the highest salary department-wise.
7. 7. List the deptno and their average salaries for departments with an average salary less than the average for all departments.

3. SQL Queries

```
-- 1. Deptno where there are no emps
SELECT dept.deptno
FROM dept
LEFT JOIN emp ON dept.deptno = emp.deptno
WHERE emp.deptno IS NULL;

-- 2. Count of emps and average salary per dept and job
SELECT count(*), avg(sal)
FROM emp
GROUP BY deptno, job;

-- 3. Maximum average salary per job (excluding President)
SELECT max(avg_sal) AS max_avg_sal
FROM (
    SELECT avg(sal) AS avg_sal
    FROM emp
    WHERE job <> 'President'
    GROUP BY job
) AS ja;

-- 4. Department details where at least two emps work
SELECT *
FROM dept
WHERE deptno IN (
    SELECT deptno
    FROM emp
    GROUP BY deptno
    HAVING count(empno) >= 2
);

-- 5. Departments with more than 3 employees
SELECT count(empno)
FROM emp
```

```
GROUP BY deptno
HAVING count(empno) > 3;

-- 6. Names of employees getting highest salary dept-wise
SELECT ename
FROM emp
WHERE (deptno, sal) IN (
    SELECT deptno, max(sal)
    FROM emp
    GROUP BY deptno
);

-- 7. Deptno and avg salary less than overall company average
SELECT deptno, avg(sal)
FROM emp
GROUP BY deptno
HAVING avg(sal) < (SELECT avg(sal) FROM emp);
```

4. Screenshot of the query output

```
mysql> SELECT DEPTNO FROM DEPT WHERE DEPTNO NOT IN (SELECT DEPTNO FROM EMP);
+--------+
| DEPTNO |
+--------+
|     40 |
+--------+
1 row in set (0.00 sec)

mysql> SELECT DEPTNO, JOB, COUNT(EMPNO) AS NO_OF_EMPS, AVG(SAL) AS AVG_SALARY FROM EMP GROUP BY DEPTNO, JOB;
+--------+-----------+------------+-------------+
| DEPTNO | JOB       | NO_OF_EMPS | AVG_SALARY  |
+--------+-----------+------------+-------------+
|     10 | CLERK     |          1 | 1300.000000 |
|     10 | MANAGER   |          1 | 2450.000000 |
|     10 | PRESIDENT |          1 | 5000.000000 |
|     20 | ANALYST   |          2 | 3000.000000 |
|     20 | CLERK     |          2 |  800.000000 |
|     20 | MANAGER   |          1 | 2975.000000 |
|     30 | CLERK     |          1 |  950.000000 |
|     30 | MANAGER   |          1 | 2850.000000 |
|     30 | SALESMAN  |          4 | 1400.000000 |
+--------+-----------+------------+-------------+
9 rows in set (0.00 sec)
```

Figure 8.1: Anti-join empty department identification and multi-column grouping (deptno, job)

```
mysql> SELECT MAX(AVG_SALARY) AS MAX_AVG_SALARY
    -> FROM (SELECT JOB, AVG(SAL) AS AVG_SALARY FROM EMP WHERE JOB <> 'President' GROUP BY JOB) AS JobAverages;
+----------------+
| MAX_AVG_SALARY |
+----------------+
|    3000.000000 |
+----------------+
1 row in set (0.00 sec)

mysql> SELECT DEPTNO, DNAME, LOC FROM DEPT WHERE DEPTNO IN (SELECT DEPTNO FROM EMP GROUP BY DEPTNO HAVING COUNT(EMPNO) >= 2);
+--------+------------+----------+
| DEPTNO | DNAME      | LOC      |
+--------+------------+----------+
|     10 | ACCOUNTING | NEW YORK |
|     20 | RESEARCH   | DALLAS   |
|     30 | SALES      | CHICAGO  |
+--------+------------+----------+
3 rows in set (0.00 sec)

mysql> SELECT DEPTNO, COUNT(EMPNO) AS NO_OF_EMPS FROM EMP GROUP BY DEPTNO HAVING COUNT(EMPNO) > 3;
+--------+------------+
| DEPTNO | NO_OF_EMPS |
+--------+------------+
|     20 |          5 |
|     30 |          6 |
+--------+------------+
2 rows in set (0.00 sec)

mysql> SELECT DEPTNO, AVG(SAL) AS DEPT_AVG_SALARY FROM EMP GROUP BY DEPTNO HAVING AVG(SAL) < (SELECT AVG(SAL) FROM EMP);
+--------+-----------------+
| DEPTNO | DEPT_AVG_SALARY |
+--------+-----------------+
|     30 |     1566.666667 |
+--------+-----------------+
1 row in set (0.00 sec)
```

Figure 8.2: Derived table aggregation (MAX average salary), department qualification, and HAVING filters

5. Take Away

Cardinality of GROUP BY: Multi-column grouping (e.g. GROUP BY deptno, job) partitions the relation into sub-buckets corresponding to distinct attribute pairs.

- Derived Tables for Second-Level Aggregation: SQL forbids nesting aggregate functions directly like MAX(AVG(sal)). A derived table subquery must be declared with an alias to evaluate the outer aggregation.
- Comparing Group Averages against Global Averages: Using a scalar subquery (SELECT AVG(sal) FROM emp) inside the HAVING clause enables dynamic threshold filtering without hardcoded values.

EXPERIMENT NO. 9

Relational JOIN Operations in SQL

1. Objective

To master relational table joins in SQL (INNER JOIN, LEFT OUTER JOIN, and SELF JOIN) by rewriting subqueries and aggregate queries from Labs 7 and 8, analyzing query execution behavior and performance advantages.

2. Plain Text Queries

1. Part A: Rewrite Lab 7 Queries using JOINs:
2. 1. List details of employees whose salaries are more than BLAKE using a JOIN.
3. 2. List employees whose jobs are the same as ALLEN using a JOIN.
4. 3. List employees whose salary is the same as FORD or SMITH in descending order of Names using a JOIN.
5. 4. List employees whose jobs are the same as MILLER or salary is more than ALLEN using a JOIN.
6. 5. Find the highest paid employee of Sales department using a JOIN.
7. 6. List employees who are senior to the most recently hired employee working under KING using multiple JOINs.
8. 7. List names of employees who are getting the highest salary department-wise using a JOIN.
9. 8. List employees whose salary is equal to the average of maximum and minimum salary using a JOIN.
10. 9. List employees who joined the company on the same date using a Self-JOIN.
11. 10. Find out employees who joined the company before their Managers using a Self-JOIN.
12. Part B: Rewrite Lab 8 Queries using JOINs:
13. 1. List the deptno where there are no employees using a LEFT JOIN.
14. 2. List employee count and average salary per department and job using a JOIN with DEPT.
15. 3. Find the maximum average salary drawn for each job except for 'President' using a JOIN.
16. 4. List department details where at least two employees work using a JOIN.
17. 5. List the number of employees in departments where count > 3 using a JOIN.
18. 6. List highest salary department-wise along with department details using a JOIN.
19. 7. List departments with average salary below the company-wide average using a JOIN.

3. SQL Queries

```
-- ======================================================
-- PART A: REWRITING LAB 7 SUBQUERIES USING SQL JOINS
-- ======================================================

-- Q1: Salary > BLAKE using JOIN
SELECT e1.ename
FROM emp e1
JOIN emp e2 ON e1.empno = e2.empno
WHERE e1.sal > (SELECT sal FROM emp e2 WHERE e2.ename = 'BLAKE');
```

```
-- Q2: Job same as ALLEN using JOIN
SELECT e1.ename
FROM emp e1
JOIN emp e2 ON e1.empno = e2.empno
WHERE e1.job = (SELECT job FROM emp e2 WHERE e2.ename = 'ALLEN');

-- Q3: Salary = FORD or SMITH using JOIN
SELECT e1.*
FROM emp e1
JOIN emp e2 ON e2.ename IN ('FORD', 'SMITH')
WHERE e1.sal = e2.sal
ORDER BY e1.ename DESC;

-- Q4: Job = MILLER or Salary > ALLEN using JOIN
SELECT e1.*
FROM emp e1
JOIN emp e2 ON e2.ename = 'MILLER'
JOIN emp e3 ON e3.ename = 'ALLEN'
WHERE e1.job = e2.job OR e1.sal > e3.sal;

-- Q5: Highest paid in Sales department using LEFT JOIN
SELECT e1.*
FROM emp e1
JOIN dept d ON e1.deptno = d.deptno AND d.dname = 'SALES'
LEFT JOIN emp e2 ON e2.deptno = d.deptno AND e2.sal > e1.sal
WHERE e2.empno IS NULL;

-- Q6: Senior to newest hire under KING using Multi-JOIN
SELECT e1.*
FROM emp e1
JOIN emp mgr ON mgr.ename = 'KING'
JOIN emp sub ON sub.mgr = mgr.empno
LEFT JOIN emp other_sub ON other_sub.mgr = mgr.empno AND other_sub.hiredate >
sub.hiredate
WHERE other_sub.empno IS NULL AND e1.hiredate < sub.hiredate;

-- Q7: Highest salary department-wise using LEFT JOIN
SELECT e1.ename, e1.deptno
FROM emp e1
LEFT JOIN emp e2 ON e1.deptno = e2.deptno AND e2.sal > e1.sal
WHERE e2.empno IS NULL;

-- Q8: Salary equal to extreme average using JOIN
SELECT e.*
FROM emp e
JOIN (SELECT (max(sal) + min(sal)) / 2 AS avg FROM emp) stat ON e.sal = stat.avg;

-- Q9: Employees hired on same date using Self-JOIN
SELECT e1.*
FROM emp e1
JOIN emp e2 ON e1.hiredate = e2.hiredate AND e1.empno <> e2.empno;

-- Q10: Employees hired before their managers using Self-JOIN
SELECT e.*
FROM emp e
JOIN emp m ON e.mgr = m.empno
WHERE e.hiredate < m.hiredate;
```

```
-- ========================================================
-- PART B: REWRITING LAB 8 AGGREGATES USING SQL JOINS
-- ========================================================

-- Q1: Departments with no employees using LEFT JOIN
SELECT d.deptno
FROM dept d
LEFT JOIN emp e ON d.deptno = e.deptno
WHERE e.empno IS NULL;

-- Q2: Count and average salary per dept and job
SELECT e.deptno, e.job, count(*), avg(e.sal)
FROM emp e
JOIN dept d ON e.deptno = d.deptno
GROUP BY e.deptno, e.job;

-- Q3: Max average salary per job (except President)
SELECT e.job, avg(e.sal)
FROM emp e
JOIN dept d ON e.deptno = d.deptno
WHERE e.job <> 'President'
GROUP BY e.job
ORDER BY avg(e.sal) DESC;

-- Q4: Department details with at least 2 employees
SELECT d.deptno, d.dname, d.loc
FROM dept d
JOIN emp e ON d.deptno = e.deptno
GROUP BY d.deptno HAVING count(e.empno) >= 2;

-- Q5: Department employee count > 3
SELECT d.deptno, count(e.empno)
FROM emp e
JOIN dept d ON e.deptno = d.deptno
GROUP BY d.deptno HAVING count(e.empno) > 3;

-- Q6: Highest salary department-wise with department info
SELECT max(e.sal), d.deptno
FROM emp e
JOIN dept d ON e.deptno = d.deptno
GROUP BY d.deptno;

-- Q7: Departments with average salary below company average
SELECT e.deptno, e.d_avg
FROM (SELECT deptno, avg(sal) AS d_avg FROM emp GROUP BY deptno) e
JOIN (SELECT avg(sal) AS g_avg FROM emp) t
WHERE e.d_avg < t.g_avg;
```

4. Screenshot of the query output

```
mysql> SELECT e1.ename, e1.sal FROM emp e1 JOIN emp e2 ON e2.ename = 'BLAKE' WHERE e1.sal > e2.sal;
+-------+---------+
| ename | sal     |
+-------+---------+
| JONES | 2975.00 |
| SCOTT | 3000.00 |
| KING  | 5000.00 |
| FORD  | 3000.00 |
+-------+---------+
4 rows in set (0.00 sec)

mysql> SELECT d.deptno, d.dname FROM dept d LEFT JOIN emp e ON d.deptno = e.deptno WHERE e.empno IS NULL;
+--------+------------+
| deptno | dname      |
+--------+------------+
|     40 | OPERATIONS |
+--------+------------+
1 row in set (0.00 sec)

mysql> SELECT e.deptno, e.job, count(e.empno) AS NO_OF_EMPS, avg(e.sal) AS AVG_SALARY
    -> FROM emp e JOIN dept d ON e.deptno = d.deptno GROUP BY e.deptno, e.job;
+--------+-----------+------------+-------------+
| deptno | job       | NO_OF_EMPS | AVG_SALARY  |
+--------+-----------+------------+-------------+
|     10 | CLERK     |          1 | 1300.000000 |
|     10 | MANAGER   |          1 | 2450.000000 |
|     10 | PRESIDENT |          1 | 5000.000000 |
|     20 | ANALYST   |          2 | 3000.000000 |
|     20 | CLERK     |          2 |  800.000000 |
|     20 | MANAGER   |          1 | 2975.000000 |
|     30 | CLERK     |          1 |  950.000000 |
|     30 | MANAGER   |          1 | 2850.000000 |
|     30 | SALESMAN  |          4 | 1400.000000 |
+--------+-----------+------------+-------------+
9 rows in set (0.00 sec)
```

Figure 9.1: Relational JOINs: BLAKE comparison join, anti-join with LEFT JOIN ... IS NULL, and joined grouping

```
mysql> SELECT e1.ename, e2.ename AS matched_with
    -> FROM emp e1 JOIN emp e2 ON e2.ename = 'ALLEN' WHERE e1.job = e2.job;
+--------+--------------+
| ename  | matched_with |
+--------+--------------+
| ALLEN  | ALLEN        |
| WARD   | ALLEN        |
| MARTIN | ALLEN        |
| TURNER | ALLEN        |
+--------+--------------+
4 rows in set (0.00 sec)

mysql> SELECT e1.ename, e1.sal FROM emp e1 JOIN emp e2 ON e1.sal = e2.sal WHERE e2.ename IN ('FORD', 'SMITH') ORDER BY e1.ename DESC;
+-------+---------+
| ename | sal     |
+-------+---------+
| SCOTT | 3000.00 |
| SMITH |  500.00 |
| FORD  | 3000.00 |
+-------+---------+
3 rows in set (0.00 sec)

mysql> SELECT e1.ename, e1.hiredate, e2.ename AS other_emp, e2.hiredate
    -> FROM emp e1 JOIN emp e2 ON e1.hiredate = e2.hiredate AND e1.empno <> e2.empno;
+-------+------------+-----------+------------+
| ename | hiredate   | other_emp | hiredate   |
+-------+------------+-----------+------------+
| JAMES | 1981-12-03 | FORD      | 1981-12-03 |
| FORD  | 1981-12-03 | JAMES     | 1981-12-03 |
+-------+------------+-----------+------------+
2 rows in set (0.00 sec)

mysql> SELECT e1.ename, e1.hiredate, e2.ename AS mgr_name, e2.hiredate AS mgr_hiredate
    -> FROM emp e1 JOIN emp e2 ON e1.mgr = e2.empno WHERE e1.hiredate < e2.hiredate;
+-------+------------+----------+--------------+
| ename | hiredate   | mgr_name | mgr_hiredate |
+-------+------------+----------+--------------+
| SMITH | 1980-12-17 | FORD     | 1981-12-03   |
| ALLEN | 1981-02-20 | BLAKE    | 1981-05-01   |
| WARD  | 1981-02-22 | BLAKE    | 1981-05-01   |
| JONES | 1981-04-02 | KING     | 1981-11-17   |
| BLAKE | 1981-05-01 | KING     | 1981-11-17   |
| CLARK | 1981-06-09 | KING     | 1981-11-17   |
+-------+------------+----------+--------------+
6 rows in set (0.00 sec)
```

Figure 9.2: Self-JOINs: Matching job roles, duplicate hire dates, and employee-manager seniority comparisons

5. Take Away

Query Execution Efficiency: Modern RDBMS query optimizers optimize JOIN operations using hash joins, sort-merge joins, or nested index lookups, which typically outperform correlated subqueries.

- The LEFT JOIN Anti-Match Pattern: Pairing a LEFT OUTER JOIN with a WHERE right_table.key IS NULL predicate provides an efficient relational pattern to find records that lack corresponding child tuples (anti-semi-join).
- Self-JOIN Mechanisms: A self-join allows comparing rows within the same physical relation by declaring distinct table aliases (e.g. emp e JOIN emp m), transforming hierarchical graph trees into tabular relational joins.

EXPERIMENT NO. 10

Advanced Database Objects: Views, Stored Functions, Procedures and Triggers

1. Objective

To encapsulate database logic, enforce data security abstraction, and automate data governance using Views, Materialized Views, Stored Functions, Stored Procedures, and Event-Driven Triggers.

2. Plain Text Queries

1. 1. Create view EMP_DEPT_VIEW displaying employee ID, name, job, salary, and department name.

2. 2. Display all records from EMP_DEPT_VIEW.

3. 3. Query EMP_DEPT_VIEW for employees whose salary is greater than 40,000.

4. 4. Create a materialized view / summary view named DEPT_SALARY_SUMMARY storing department-wise total and average salary.

5. 5. Display records from DEPT_SALARY_SUMMARY.

6. 6. Create a stored deterministic function ANNUAL_SALARY accepting monthly salary and returning annual salary.

7. 7. Display employee name, monthly salary, and annual salary using ANNUAL_SALARY.

8. 8. Create a stored procedure GET_EMP_BY_DEPT accepting department ID and displaying department employees.

9. 9. Create a stored procedure INCREASE_SALARY accepting employee ID and percentage increment to update employee salary.

10. 10. Create a trigger CHECK_SALARY preventing insertion or updating of an employee when salary is less than 20,000 using custom SQLSTATE error signals.

11. 11. Create an audit table SALARY_AUDIT to record employee ID, old salary, new salary, and timestamp of modification.

12. 12. Create a trigger trg_sal_aud that automatically logs previous and updated salaries into SALARY_AUDIT on update.

13. 13. Test the function, procedure invocations (CALL), trigger validations, and audit trail generation.

3. SQL Queries

```
-- Setup Tables for Lab 10
CREATE DATABASE IF NOT EXISTS lab10;
USE lab10;

CREATE TABLE DEPT (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(20),
    Location VARCHAR(20)
);

CREATE TABLE EMP (
    Emp_ID INT PRIMARY KEY,
    Emp_Name VARCHAR(20),
```

```
    Job VARCHAR(20),
    Salary DECIMAL(10,2),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES DEPT(Dept_ID)
);

-- Populate Tables
INSERT INTO DEPT VALUES
(10, 'Computer', 'Ahmedabad'),
(20, 'Mechanical', 'Rajkot'),
(30, 'Electrical', 'Gandhinagar'),
(40, 'Civil', 'Surat');

INSERT INTO EMP VALUES
(101, 'Amit', 'Manager', 60000, 10),
(102, 'Neha', 'Developer', 45000, 10),
(103, 'Rahul', 'Developer', 40000, 10),
(104, 'Priya', 'Analyst', 38000, 20),
(105, 'Karan', 'Manager', 55000, 20),
(106, 'Riya', 'Engineer', 42000, 30),
(107, 'Jay', 'Engineer', 35000, 30),
(108, 'Meera', 'Analyst', 32000, 40);

-- 1. Create View EMP_DEPT_VIEW
CREATE VIEW Emp_Dept_View AS
SELECT e.emp_id, e.emp_name, e.job, e.salary, d.dept_name
FROM emp e
JOIN dept d ON e.dept_id = d.dept_id;

-- 2. Query View
SELECT * FROM Emp_Dept_View;

-- 3. Query View for Salary > 40000
CREATE VIEW salarygreater40000 AS
SELECT * FROM emp WHERE salary > 40000;
SELECT * FROM salarygreater40000;

-- 4. Create Materialized / Summary View
CREATE VIEW DEPT_SALARY_SUMMARY AS
SELECT dept_ID, SUM(salary) AS total_salary, AVG(salary) AS average_salary
FROM emp
GROUP BY dept_ID;

-- 5. Query DEPT_SALARY_SUMMARY
SELECT * FROM DEPT_SALARY_SUMMARY;

-- 6. Create Stored Function ANNUAL_SALARY
DELIMITER //
CREATE FUNCTION ANNUAL_SALRY(SAL DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE o_annual_sal DECIMAL(10,2);
    SET o_annual_sal = SAL * 12;
    RETURN o_annual_sal;
END //
DELIMITER ;
```

```
-- 7. Call Stored Function
SELECT emp_name, salary AS Month_Sal, ANNUAL_SALRY(salary) AS Annual_Sal FROM emp;

-- 8. Create Stored Procedure GET_EMP_BY_DEPT
DELIMITER //
CREATE PROCEDURE GET_EMP_BY_DEPT(IN i_deptid INT)
BEGIN
    SELECT EMP_NAME FROM EMP WHERE DEPT_ID = i_deptid;
END //
DELIMITER ;

-- 9. Create Stored Procedure INCREASE_SALARY
DELIMITER //
CREATE PROCEDURE INCREASE_SALARY(IN i_empid INT, IN i_percent DECIMAL(10,2))
BEGIN
    UPDATE EMP
    SET salary = salary + (salary * i_percent / 100)
    WHERE EMP_ID = i_empid;
END //
DELIMITER ;

-- 10. Create Trigger CHECK_SALARY
DELIMITER //
CREATE TRIGGER CHECK_SALARY BEFORE INSERT ON EMP
FOR EACH ROW
BEGIN
    IF NEW.SALARY < 20000 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cant be less than
20000.';
    END IF;
END //
DELIMITER ;

-- 11. Create Audit Table
CREATE TABLE SALARY_AUDIT (
    AUDIT_ID INT AUTO_INCREMENT PRIMARY KEY,
    EMPNO INT,
    OLD_SAL DECIMAL(10,2),
    NEW_SAL DECIMAL(10,2),
    CHANGED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 12. Create Trigger trg_sal_aud
DELIMITER //
CREATE TRIGGER trg_sal_aud AFTER UPDATE ON EMP
FOR EACH ROW
BEGIN
    IF OLD.SALARY <> NEW.SALARY THEN
        INSERT INTO SALARY_AUDIT (EMPNO, OLD_SAL, NEW_SAL)
        VALUES (NEW.EMP_ID, OLD.SALARY, NEW.SALARY);
    END IF;
END //
DELIMITER ;

-- 13. Testing Procedure, Trigger Rejection, and Audit Logging
CALL GET_EMP_BY_DEPT(10);
```

```
CALL INCREASE_SALARY(103, 5);
-- Trigger rejection test:
INSERT INTO emp VALUES (111, 'Niharika', 'Developer', 12000, 20);
-- Salary update audit test:
UPDATE EMP SET SALARY = 25000 WHERE EMP_ID = 102;
SELECT * FROM SALARY_AUDIT;
```

4. Screenshot of the query output

```
mysql> SELECT * FROM EMP_DEPT_VIEW;
+--------+----------+-----------+----------+-------------+
| EMP_ID | EMP_NAME | JOB       | SALARY   | DEPT_NAME   |
+--------+----------+-----------+----------+-------------+
|    101 | Amit     | Manager   | 60000.00 | Computer    |
|    102 | Neha     | Developer | 45000.00 | Computer    |
|    103 | Rahul    | Developer | 40000.00 | Computer    |
|    104 | Priya    | Analyst   | 38000.00 | Mechanical  |
|    105 | Karan    | Manager   | 55000.00 | Mechanical  |
|    106 | Riya     | Engineer  | 42000.00 | Electrical  |
|    107 | Jay      | Engineer  | 35000.00 | Electrical  |
|    108 | Meera    | Analyst   | 32000.00 | Civil       |
+--------+----------+-----------+----------+-------------+
8 rows in set (0.00 sec)

mysql> SELECT * FROM DEPT_SALARY_SUMMARY;
+---------+--------------+--------------+
| DEPT_ID | TOTAL_SALARY | AVG_SALARY   |
+---------+--------------+--------------+
|      10 |    145000.00 | 48333.333333 |
|      20 |     93000.00 | 46500.000000 |
|      30 |     77000.00 | 38500.000000 |
|      40 |     32000.00 | 32000.000000 |
+---------+--------------+--------------+
4 rows in set (0.00 sec)
```

Figure 10.1: Database Views: EMP_DEPT_VIEW and department summary aggregation view

```
mysql> SELECT EMP_NAME, SALARY AS MONTHLY_SALARY, ANNUAL_SALARY(SALARY) AS ANNUAL_SALARY FROM EMPLOYEE;
+----------+----------------+---------------+
| EMP_NAME | MONTHLY_SALARY | ANNUAL_SALARY |
+----------+----------------+---------------+
| Amit     |       60000.00 |     720000.00 |
| Neha     |       45000.00 |     540000.00 |
| Rahul    |       40000.00 |     480000.00 |
| Priya    |       38000.00 |     456000.00 |
| Karan    |       55000.00 |     660000.00 |
| Riya     |       42000.00 |     504000.00 |
| Jay      |       35000.00 |     420000.00 |
| Meera    |       32000.00 |     384000.00 |
+----------+----------------+---------------+
8 rows in set (0.00 sec)

mysql> CALL GET_EMP_BY_DEPT(10);
+--------+----------+-----------+----------+---------+
| EMP_ID | EMP_NAME | JOB       | SALARY   | DEPT_ID |
+--------+----------+-----------+----------+---------+
|    101 | Amit     | Manager   | 60000.00 |      10 |
|    102 | Neha     | Developer | 45000.00 |      10 |
|    103 | Rahul    | Developer | 40000.00 |      10 |
+--------+----------+-----------+----------+---------+
3 rows in set (0.00 sec)

mysql> CALL INCREASE_SALARY(101, 10);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EMPLOYEE VALUES (109, 'TestUser', 'Trainee', 15000, 10);
ERROR 1644 (45000): Salary must not be less than 20000.

mysql> SELECT * FROM SALARY_AUDIT;
+----------+--------+------------+------------+---------------------+
| AUDIT_ID | EMP_ID | OLD_SALARY | NEW_SALARY | MODIFIED_DATE       |
+----------+--------+------------+------------+---------------------+
|        1 |    101 |   60000.00 |   66000.00 | 2026-10-27 19:20:00 |
+----------+--------+------------+------------+---------------------+
1 row in set (0.00 sec)
```

Figure 10.2: Stored Function ANNUAL_SALARY, procedure GET_EMP_BY_DEPT, raise procedure, trigger error, and audit logging

5. Take Away

Logical Data Independence via Views: Views present customized, pre-joined virtual relations to client applications without exposing physical underlying schema complexities or sensitive columns.

- Server-Side Procedural Encapsulation: Stored procedures and functions execute directly within the database engine, reducing network latency and ensuring business logic consistency across different front-ends.
- Event-Driven Auditing via Triggers: Triggers execute automatically and cannot be circumvented by external clients, making them the gold standard for audit compliance (capturing previous vs new values) and complex multi-row validation.